ENGINEERING COLLABORATION CHARTER

AegisSRE Team Delivery Plan

An equal, evidence-driven process for Guvvala Karthik and Trivedi to build a production-grade ML, deep learning, transformer, generative AI and agentic SRE platform.

CANONICAL REPOSITORY

github.com/guvvalakarthik/aegis-sre

OWNERSHIP

100 effort points each

DELIVERY

One task, one branch, one reviewed PR

VERSION

1.0 | 1 August 2026

Contents

- AegisSRE team collaboration plan
  - 1. Objective
  - 2. How the repository appears in both accounts
  - 3. Equal ownership: 100 points each
    - Karthik - Platform and ML lead
    - Trivedi - AI systems and product lead
  - 4. Shared responsibilities
  - 5. Delivery sequence
  - 6. One task, one branch, one reviewed PR
  - 7. Definition of done
  - 8. GitHub governance
  - 9. Correct contribution credit
  - 10. Communication cadence
  - 11. v1.0 release evidence
  - 12. Immediate actions

AegisSRE team collaboration plan

Project: AegisSRE
Canonical repository: https://github.com/guvvalakarthik/aegis-sre
Contributors: Guvvala Karthik and Trivedi
Status: Account setup awaits Trivedi's exact GitHub username
Version: 1.0 - 2026-08-01

1. Objective

Build AegisSRE as a production-grade portfolio and research project covering Kubernetes observability, incident simulation, classical ML, deep learning, transformers, hybrid RAG, agentic workflows, remediation safety, evaluation, MLOps and cloud deployment.

A capability is complete only when it is implemented, reproducible, measured, documented and demonstrated. Technology names without executable evidence do not count.

2. How the repository appears in both accounts

Use one canonical upstream repository and one personal fork:

- **Karthik:** owns `guvvalakarthik/aegis-sre`, the canonical upstream.
- **Trivedi:** receives write access and owns a personal `aegis-sre` fork.
- **Result:** the original appears in Karthik's repositories, the fork appears in Trivedi's repositories, and accepted work is merged into one authoritative `main`.
- **Identity:** each person uses an individual verified GitHub email. Never share passwords, tokens, SSH keys, cloud credentials or company identities.

> Required input: replace `TRIVEDI_GITHUB_USERNAME` with Trivedi's exact GitHub login. Do not guess it.

Karthik sends the collaborator invitation:

```
gh api --method PUT   repos/guvvalakarthik/aegis-sre/collaborators/TRIVEDI_GITHUB_USERNAME   -f
permission=push
```

After accepting, Trivedi creates and configures the fork:

```
gh repo fork guvvalakarthik/aegis-sre --clone=false
git clone https://github.com/TRIVEDI_GITHUB_USERNAME/aegis-sre.git
cd aegis-sre
git remote add upstream https://github.com/guvvalakarthik/aegis-sre.git
git fetch upstream
git remote -v
```

The fork uses `origin`; the canonical repository uses `upstream`.

3. Equal ownership: 100 points each

Effort points represent expected engineering difficulty, not lines of code. Any reassignment must preserve the final 100/100 split.

Karthik - Platform and ML lead

ID	Work package	Task branch	Points	Acceptance evidence
K1	API, CI and security foundation	`agent/project-foundation`	10	Merged PR, typing, tests, container
K2	Kubernetes and OpenTelemetry testbed	`karthik/02-observability-testbed`	15	Pinned chart and live telemetry
K3	Telemetry ingestion and schemas	`karthik/03-telemetry-pipeline`	15	Normalized signals with provenance
K4	Reproducible incident laboratory	`karthik/04-incident-lab`	15	Ground truth, seeds and replay
K5	Classical ML baselines	`karthik/05-classical-ml`	15	Rules, Isolation Forest and XGBoost
K6	Model-serving and control-plane APIs	`karthik/06-model-serving`	10	Versioned, authenticated inference
K7	MLOps, model registry and drift	`karthik/07-mlops`	10	Lineage, promotion and rollback
K8	Cloud infrastructure and deployment	`karthik/08-cloud-deployment`	10	Reproducible staging environment
	Total		**100**	

Trivedi - AI systems and product lead

ID	Work package	Task branch	Points	Acceptance evidence
T1	Dataset validation and feature contracts	`trivedi/01-data-quality`	10	Leakage checks and time-aware splits
T2	Deep-learning anomaly baselines	`trivedi/02-deep-learning`	15	Autoencoder and TCN/LSTM benchmark
T3	Time-series and log transformers	`trivedi/03-transformers`	20	PatchTST, log encoder and ablations
T4	Hybrid RAG and evidence citations	`trivedi/04-hybrid-rag`	15	Hybrid retrieval and retrieval metrics
T5	Topology and incident knowledge graph	`trivedi/05-knowledge-graph`	10	Service, trace and deploy relationships
T6	Typed agent workflow	`trivedi/06-agent-workflow`	15	State, budgets, retries and evidence
T7	Safety critic and remediation sandbox	`trivedi/07-safety-sandbox`	10	Policy, approval, canary and rollback
T8	Incident and evaluation dashboard	`trivedi/08-dashboard`	5	Topology, evidence and benchmark UI
	Total		**100**	

4. Shared responsibilities

Both contributors jointly own:

- Architecture decisions and threat modelling
- Incident ground-truth definitions

- Benchmark design and result interpretation
- Review of every pull request from the other contributor
- Adversarial and safety evaluation
- Final system integration
- Technical report, demo video and interview preparation

Karthik must be able to explain Trivedi's AI architecture. Trivedi must be able to explain Karthik's platform, data and MLOps architecture.

5. Delivery sequence

Sprint	Karthik	Trivedi	Integration gate
0	Repository and rules	Fork, identity and environment	Both can branch and run CI
1	Kubernetes/OTel testbed	Data-quality contracts	Stable telemetry and schemas
2	Telemetry pipeline and incident lab	DL anomaly baselines	Versioned dataset v1
3	Classical ML baselines	Transformers	Reproducible benchmark v1
4	Model-serving APIs	Hybrid RAG and graph	Evidence-linked diagnosis
5	MLOps and drift	Agent workflow and safety	Controlled incident recovery
6	Cloud staging deployment	Dashboard	End-to-end staging demo
7	Cost, load and reliability tests	UX and evaluation views	Release candidate
8	Joint report, demo and defence	Joint report, demo and defence	v1.0 release

A downstream sprint begins only after its integration gate passes on canonical `main`.

6. One task, one branch, one reviewed PR

Never implement directly on `main`.

Start a task from canonical main:

```
git fetch upstream
git switch main
git merge --ff-only upstream/main
git switch -c trivedi/03-transformers
```

Karthik uses a `karthik/` prefix. Agent-assisted branches use `agent/` and still require a human owner and reviewer.

Before pushing:

```
ruff format --check .
ruff check .
mypy src
pytest
git diff --check
```

Push and open a pull request:

```
git push -u origin trivedi/03-transformers
gh pr create --repo guvvalakarthik/aegis-sre   --base main   --head TRIVEDI_GITHUB_USERNAME:trivedi/03-
transformers
```

Every pull request must:

- Link one primary issue/work package.
- Explain what changed, why and relevant trade-offs.
- Include real tests tied to acceptance criteria; no illustrative test cases.
- Include benchmark artifact IDs for model or data changes.
- Describe security, failure and rollback behavior.
- Receive approval from the other contributor.
- Pass every required CI check and resolve every conversation.
- Use squash merge after the collaboration ruleset is enabled.
- Preserve the task branch until the sprint release tag.

7. Definition of done

- [ ] Acceptance criteria are executable and pass.
- [ ] Formatting, linting and strict type checking pass.
- [ ] Unit, integration and end-to-end tests cover real behavior.
- [ ] Data and model lineage is recorded.
- [ ] Evaluation uses held-out, time-aware data where applicable.
- [ ] Metrics are generated by scripts, not typed into documentation.
- [ ] Secrets and personal data are absent from code, logs and artifacts.
- [ ] Failure behavior and rollback are tested.
- [ ] Documentation matches implemented behavior and limitations.
- [ ] The other contributor approved the pull request.
- [ ] CI passed on the exact commit merged to `main`.

Placeholders, fabricated metrics, fake model output, empty abstractions, direct pushes and tests that only assert hard-coded examples are incomplete.

8. GitHub governance

Create a GitHub Project with:

1. Backlog

2. Ready
3. In progress
4. Review
5. Validation
6. Done

Every issue records owner, package ID, points, dependencies, acceptance criteria, required tests, documentation, security considerations and required evidence.

Enable a `main` ruleset that:

- Requires a pull request and one reciprocal approval
- Dismisses stale approvals
- Requires all CI checks and conversation resolution
- Blocks force pushes and branch deletion
- Allows emergency bypass only for a documented incident

9. Correct contribution credit

- Configure repository-local name and a verified GitHub email.
- The person doing the substantive work authors the package commits.
- Use `Co-authored-by` only when both people actually wrote the commit.
- Merge work into canonical `main`.
- Never commit from shared, company or unrelated identities.
- Keep Trivedi's fork synchronized with `upstream/main`.

```
git config --local user.name "TRIVEDI_DISPLAY_NAME"
git config --local user.email "TRIVEDI_VERIFIED_GITHUB_EMAIL"
```

10. Communication cadence

- **Daily async:** completed, next action, blocker and evidence link.
- **Twice weekly:** integrate both active branches in a clean environment.
- **Weekly:** architecture/security review and ADR updates.
- **Sprint close:** tag source, dataset, models and release notes.
- **One-day blocker rule:** a blocker lasting one workday becomes a GitHub issue.

GitHub Issues, ADRs and PR comments are the source of truth. Chat coordinates work but does not replace permanent engineering evidence.

11. v1.0 release evidence

- Reproducible local and staging deployment
- At least 25 deterministic incident scenarios

- Dataset card and model cards
- ML, DL and transformer comparisons
- Retrieval and citation evaluation
- Root-cause Top-1 and Top-3 metrics
- Recovery success and unsafe-action metrics
- Latency and cost measurements
- Threat model and adversarial evaluation
- Architecture diagrams and runbooks
- Public dashboard or recorded end-to-end demo
- Technical report with honest limitations
- Tagged source, dataset version and model versions

12. Immediate actions

1. Provide Trivedi's exact GitHub username.
2. Send and accept write-collaborator access.
3. Create Trivedi's personal fork.
4. Configure verified repository-local identities.
5. Enable the canonical `main` ruleset and required CI.
6. Create the Project board and all K/T work-package issues.
7. Assign K2 to Karthik and T1 to Trivedi.
8. Hold the first integration review after both first PRs pass.
